

Lab 5: Implementation of Page Replacement Algorithms (FIFO, LRU, Optimal and Clock)

Theory

A page replacement algorithm is a method used by an operating system to decide which page should be removed from main memory when a new page needs to be loaded and all available memory frames are occupied. In a virtual memory system, processes are divided into pages, and only some of these pages are kept in physical memory at a time. When a required page is not present in memory, a page fault occurs. If there is no free frame, the operating system must replace one of the existing pages.

Different algorithms use different criteria for selecting a page for replacement. In this lab, FIFO, LRU, Optimal, and Clock page replacement algorithms are studied and implemented.

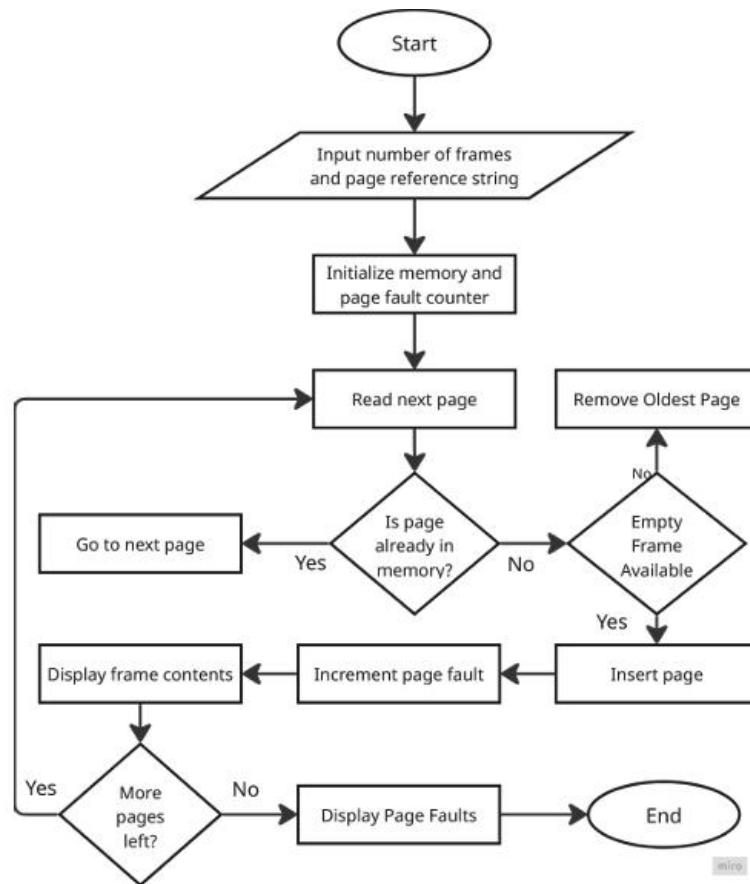
First-In, First-Out (FIFO)

First-In, First-Out (FIFO) is a page replacement algorithm that replaces the page that has been in memory for the longest time. It follows the principle that the first page loaded into memory should be the first page removed. FIFO maintains the pages according to their arrival order, usually using a queue.

When a page is referenced, the system first checks whether it is already present in memory. If it is present, a page hit occurs and no replacement is required. If it is not present, a page fault occurs. If a free frame exists, the page is loaded into it. Otherwise, the oldest page is removed and the new page is inserted.

Working:

1. Check whether the requested page is in memory.
2. If present, count it as a page hit.
3. If absent, generate a page fault.
4. If a free frame exists, load the page.
5. Otherwise, replace the oldest page.
6. Continue until all page references are processed.



Advantages:

- Simple and easy to implement.
- Requires little additional information about page usage.

Disadvantages:

- May replace a frequently used page.
- Can produce a relatively high number of page faults.

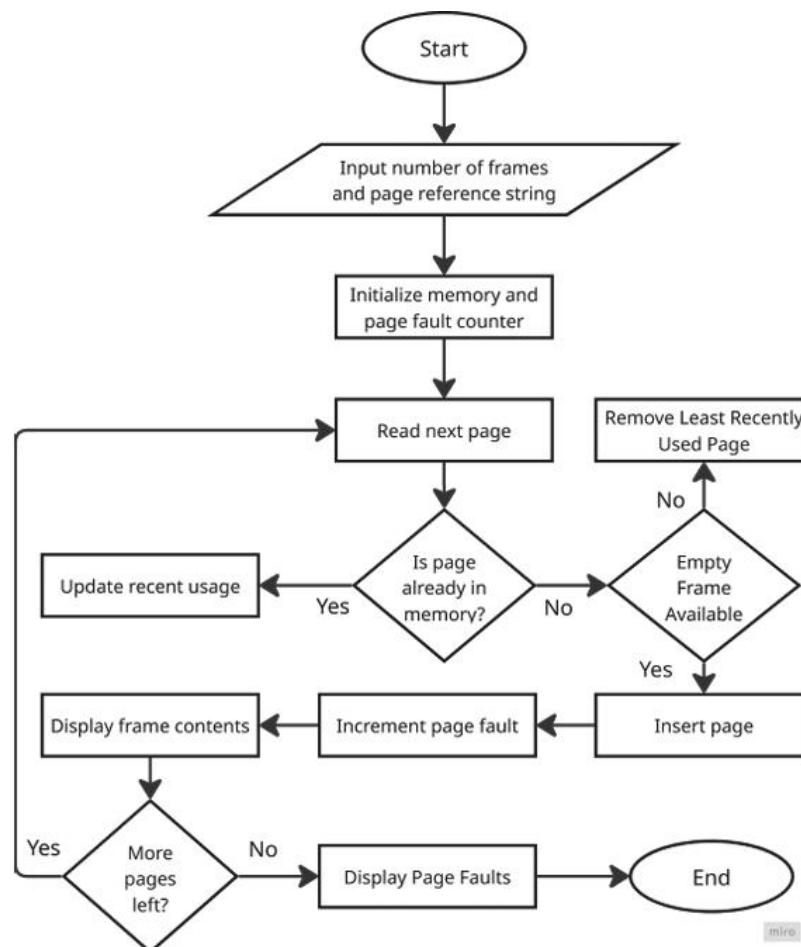
Least Recently Used (LRU)

Least Recently Used (LRU) is a page replacement algorithm that replaces the page that has not been used for the longest period of time. It is based on the idea that pages used recently are more likely to be used again, while pages that have not been used for a long time are less likely to be needed immediately.

When a page reference occurs, the system checks whether the page is already in memory. If the page is present, its recent-use information is updated. If the page is absent and all frames are occupied, the page that has remained unused for the longest time is selected for replacement.

Working:

1. Check whether the requested page is in memory.
2. If present, record its recent use.
3. If absent, generate a page fault.
4. If a free frame exists, load the page.
5. Otherwise, identify the least recently used page.
6. Replace it with the required page.

**Advantages:**

- Generally, makes better replacement decisions than FIFO
- Uses recent page access information to avoid replacing recently used pages.

Disadvantages:

- Requires additional information to track page usage.
- Implementing exact LRU can introduce additional overhead.

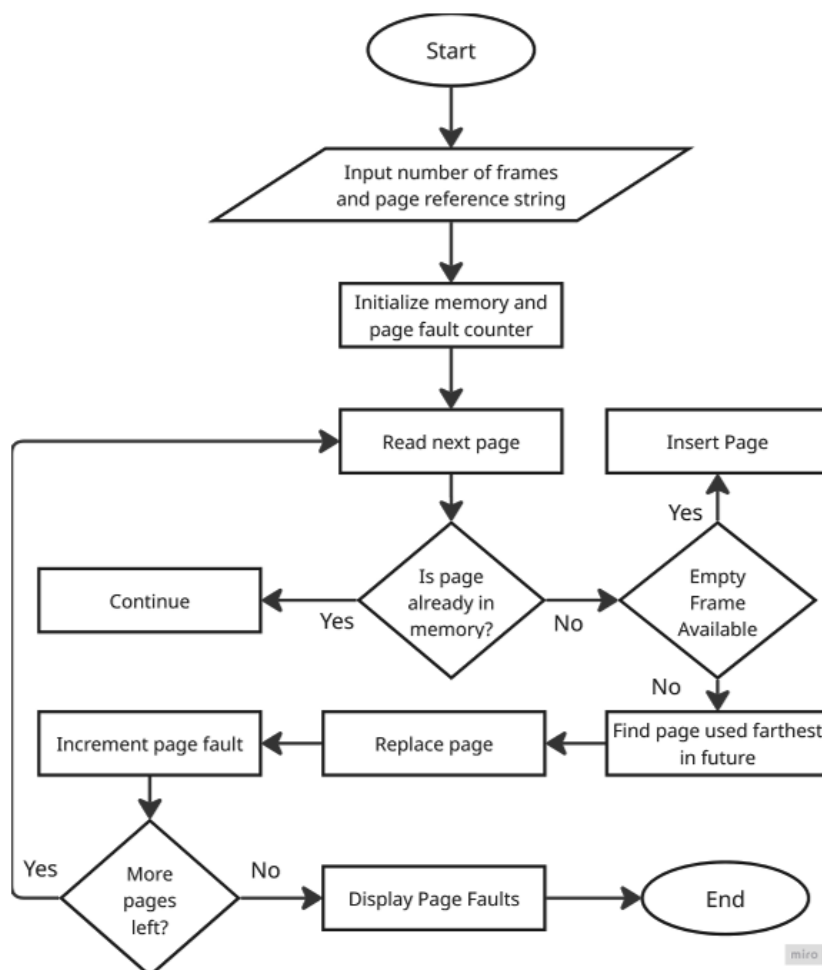
Optimal Page Replacement

Optimal Page Replacement is a theoretical page replacement algorithm that replaces the page whose next use occurs farthest in the future. When a page fault occurs and all frames are occupied, the algorithm examines the future page references and selects the page that will not be needed for the longest time.

For example, if several pages are currently in memory, the algorithm looks ahead in the reference string to determine when each page will next be required. A page that will be needed much later, or will not be referenced again, is selected for replacement.

Working:

1. Check whether the requested page is in memory.
2. If present, count it as a page hit.
3. If absent, generate a page fault.
4. If a free frame exists, load the page.
5. Otherwise, examine future references of the pages in memory.
6. Replace the page whose next reference is farthest in the future.



Advantages:

- Produces the minimum possible number of page faults for a given reference string.
- Provides a standard for comparing other page replacement algorithms.

Disadvantages:

- Requires knowledge of future page references.
- Cannot be implemented exactly in a practical operating system because future references are not known in advance.

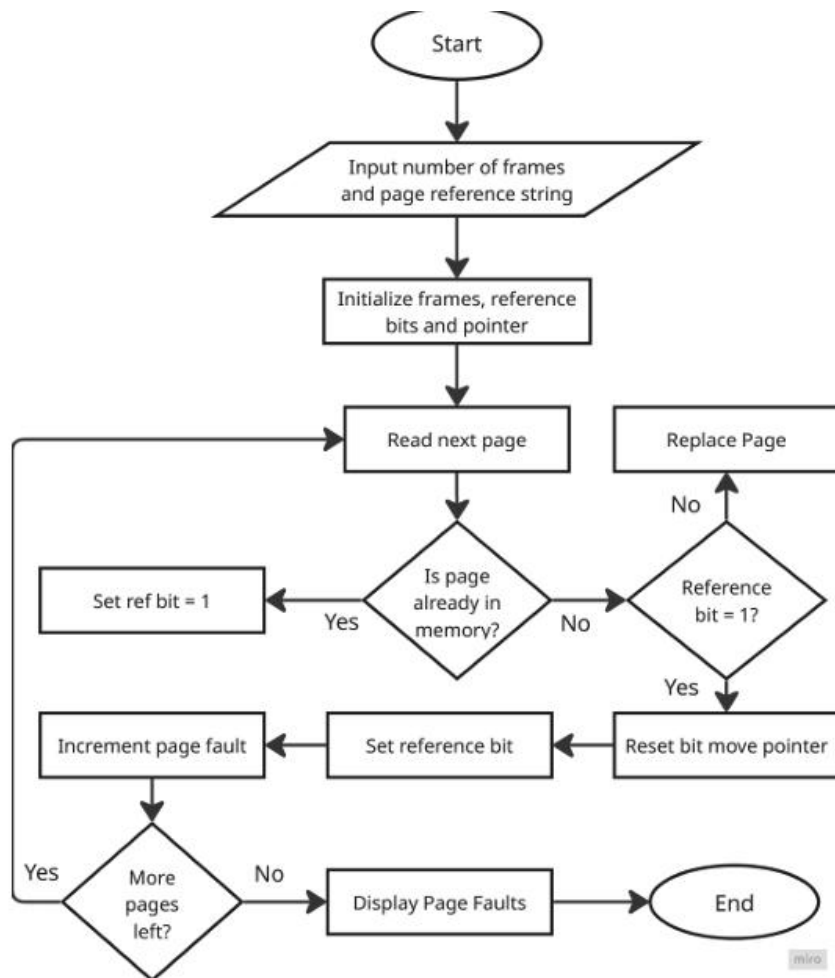
Clock Page Replacement

Clock is a page replacement algorithm that provides an approximation of the Least Recently Used (LRU) algorithm. It uses a circular arrangement of memory frames and a reference bit associated with each page. A pointer, commonly called the clock hand, moves through the frames to identify a suitable page for replacement.

When a page is referenced, its reference bit is set to 1. During replacement, the clock hand checks the reference bit of each page. If the bit is 0, that page can be replaced. If the bit is 1, the algorithm changes it to 0 and moves the clock hand to the next frame. The search continues until a page with a reference bit of 0 is found.

Working:

1. Check whether the requested page is in memory.
2. If present, set its reference bit to 1.
3. If a page fault occurs, check the page indicated by the clock hand.
4. If its reference bit is 0, replace that page.
5. If its reference bit is 1, change it to 0 and move the hand forward.
6. Continue until a replacement page is found.



Advantages:

- Requires less overhead than implementing exact LRU.
- Efficiently uses a reference bit to identify recently used pages.

Disadvantages:

- It is only an approximation of LRU.
- Its replacement decision can depend on the position of the clock hand.

Source Code

1. First In First Out (FIFO)

```
#include <stdio.h>

int main() {
    int stream[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int frames = 3, pages, faults = 0, hits = 0, pointer = 0;
    int temp[frames], i, j, found;

    pages = sizeof(stream) / sizeof(stream[0]);

    printf("Incoming\tFrame 1\t\tFrame 2\t\tFrame 3");

    for (i = 0; i < frames; i++) {
        temp[i] = -1;
    }

    for (i = 0; i < pages; i++) {
        found = 0;

        for (j = 0; j < frames; j++) {
            if (stream[i] == temp[j]) {
                found = 1;
                hits++;
                break;
            }
        }

        if (found == 0) {
            temp[pointer] = stream[i];
            pointer = (pointer + 1) % frames;
            faults++;
        }
    }
}
```

```

printf("\n%d\t\t", stream[i]);

for (j = 0; j < frames; j++) {
    if (temp[j] != -1)
        printf("%d\t\t", temp[j]);
    else
        printf("-\t\t");
}
}

printf("\n\nTotal Page Faults: %d\n", faults);
printf("Page Hits: %d", hits);
return 0;
}

```

2. Least Recently Used (LRU)

```
#include <stdio.h>
#include <limits.h>

int checkHit(int page, int temp[], int occupied) {
    int i;
    for (i = 0; i < occupied; i++) {
        if (page == temp[i])
            return 1;
    }
    return 0;
}

void printFrame(int temp[], int occupied) {
    int i;
    for (i = 0; i < occupied; i++)
        printf("%d\t", temp[i]);
}

int main() {
    int stream[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int frames = 3, pages, faults = 0, hits = 0;
    int temp[frames], distance[frames];
    int occupied = 0;
    int i, j, k;
    int max, index;

    pages = sizeof(stream) / sizeof(stream[0]);
    printf("Incoming\tFrame 1\t\tFrame 2\t\tFrame 3\n");

    for (i = 0; i < pages; i++) {
        printf("%d\t", stream[i]);
        if (checkHit(stream[i], temp, occupied)) {
            hits++;

```

```

        printFrame(temp, occupied);
    }
    else if (occupied < frames) {
        temp[occupied] = stream[i];
        occupied++;
        faults++;
        printFrame(temp, occupied);
    }
    else {
        max = INT_MIN;
        for (j = 0; j < frames; j++) {
            distance[j] = 0;

            for (k = i - 1; k >= 0; k--) {
                distance[j]++;

                if (temp[j] == stream[k])
                    break;
            }
            if (distance[j] > max) {
                max = distance[j];
                index = j;
            }
        }
        temp[index] = stream[i];
        faults++;
        printFrame(temp, occupied);
    }
    printf("\n");
}
printf("\nPage Faults: %d\n", faults);
printf("Page Hits: %d", hits);
return 0;
}

```

3. Optimal Page Replacement

```
#include <stdio.h>
#include <limits.h>

int checkHit(int page, int temp[], int occupied) {
    int i;

    for (i = 0; i < occupied; i++) {
        if (page == temp[i])
            return 1;
    }
    return 0;
}

void printFrame(int temp[], int occupied) {
    int i;
    for (i = 0; i < occupied; i++)
        printf("%d\t\t", temp[i]);
}

int main() {
    int stream[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int frames = 3, pages, faults = 0, hits = 0;
    int temp[3], distance[3];
    int occupied = 0;
    int i, j, k;
    int max, index = 0;

    pages = sizeof(stream) / sizeof(stream[0]);
    printf("Incoming\tFrame 1\t\tFrame 2\t\tFrame 3\n");
    for (i = 0; i < pages; i++) {
        printf("%d\t\t", stream[i]);
        if (checkHit(stream[i], temp, occupied)) {
            hits++;

```

```

        printFrame(temp, occupied);
    }
    else if (occupied < frames) {
        temp[occupied] = stream[i];
        occupied++;
        faults++;
        printFrame(temp, occupied);
    }
    else {
        max = INT_MIN;
        for (j = 0; j < frames; j++) {
            distance[j] = INT_MAX;

            for (k = i + 1; k < pages; k++) {
                if (temp[j] == stream[k]) {
                    distance[j] = k - i;
                    break;
                }
            }
            if (distance[j] > max) {
                max = distance[j];
                index = j;
            }
        }
        temp[index] = stream[i];
        faults++;
        printFrame(temp, occupied);
    }
    printf("\n");
}
printf("\nPage Faults: %d\n", faults);
printf("Page Hits:  %d\n", hits);
return 0;
}

```

4. Clock Page Replacement

```
#include <stdio.h>
```

```
int checkHit(int page, int temp[], int occupied) {
    int i;
    for (i = 0; i < occupied; i++) {
        if (page == temp[i])
            return 1;
    }
    return 0;
}

void printFrame(int temp[], int occupied) {
    int i;
    for (i = 0; i < occupied; i++)
        printf("%d\t\t", temp[i]);
}

int main() {
    int stream[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int frames = 3, pages, faults = 0, hits = 0;
    int temp[frames], bit[frames];
    int occupied = 0, pointer = 0;
    int i, j, found;
    pages = sizeof(stream) / sizeof(stream[0]);
    for (i = 0; i < frames; i++)
        bit[i] = 0;

    printf("Incoming\tFrame 1\t\tFrame 2\t\tFrame 3\n");
    for (i = 0; i < pages; i++) {
        printf("%d\t\t", stream[i]);

        found = 0;
        for (j = 0; j < occupied; j++) {
            if (stream[i] == temp[j]) {
                bit[j] = 1;
```

```

        found = 1;
        break;
    }
}

if (found == 1) {
    hits++;
    printFrame(temp, occupied);
}
else if (occupied < frames) {
    temp[occupied] = stream[i];
    bit[occupied] = 1;
    occupied++;
    faults++;
    printFrame(temp, occupied);
}
else {
    while (bit[pointer] == 1) {
        bit[pointer] = 0;
        pointer = (pointer + 1) % frames;
    }
    temp[pointer] = stream[i];
    bit[pointer] = 1;
    pointer = (pointer + 1) % frames;
    faults++;
    printFrame(temp, occupied);
}
printf("\n");
}

printf("\nPage Faults: %d\n", faults);
printf("Page Hits: %d\n", hits);
return 0;
}

```

Output

FIFO

D:\Prasoon Raj Shakya\BSc. C

×

+

▼

Incoming	Frame 1	Frame 2	Frame 3
7	7	—	—
0	7	0	—
1	7	0	1
2	2	0	1
0	2	0	1
3	2	3	1
0	2	3	0
4	4	3	0
2	4	2	0
3	4	2	3
0	0	2	3
3	0	2	3
2	0	2	3

Total Page Faults: 10
Page Hits: 3

LRU

D:\Prasoon Raj Shakya\BSc. C

×

+

▼

Incoming	Frame 1	Frame 2	Frame 3
7	7		
0	7	0	
1	7	0	1
2	2	0	1
0	2	0	1
3	2	0	3
0	2	0	3
4	4	0	3
2	4	0	2
3	4	3	2
0	0	3	2
3	0	3	2
2	0	3	2

Page Faults: 9
Page Hits: 4

OPR

D:\Prasoon Raj Shakya\BSc. C

×

+

▾

Incoming	Frame 1	Frame 2	Frame 3
7	7		
0	7	0	
1	7	0	1
2	2	0	1
0	2	0	1
3	2	0	3
0	2	0	3
4	2	4	3
2	2	4	3
3	2	4	3
0	2	0	3
3	2	0	3
2	2	0	3

Page Faults: 7

Page Hits: 6

Clock

D:\Prasoon Raj Shakya\BSc. C

×

+

▾

Incoming	Frame 1	Frame 2	Frame 3
7	7		
0	7	0	
1	7	0	1
2	2	0	1
0	2	0	1
3	2	0	3
0	2	0	3
4	4	0	3
2	4	2	3
3	4	2	3
0	4	2	0
3	3	2	0
2	3	2	0

Page Faults: 9

Page Hits: 4

Conclusion

In this lab, FIFO, LRU, Optimal, and Clock page replacement algorithms were studied and implemented to understand how an operating system manages pages when all available memory frames are occupied. The implementation provided practical understanding of page faults and how different algorithms determine which page should be replaced.

It was learned that FIFO uses the order of page arrival, LRU considers recent page usage, Optimal considers future page references, and Clock uses reference bits to approximate LRU. The lab helped in understanding the working, advantages, and limitations of each algorithm and their role in managing physical memory efficiently.